

Recipe Discovery System — ITNE352 Project

Project Description

this project is basically a client and server system, the client is the user side where you can search and browse recipes, and the server is the one that connects to TheMealDB website and gets the data for you. so the client talks to the server and the server talks to the recipe website and sends everything back to the client.

Semester

S2 2025–2026

Group

Field	Details
Group Name	B4
Course Code	ITNE352 — Network Programming
Section	2
Student 1	Layla Hussain Ali — 202307679
Student 2	Maryam Ahmed Jalal — 202209481

Table of Contents

1. Project Description
 2. Semester
 3. Group
 4. Requirements
 5. How To Run
 6. The Scripts
 7. Additional Concept — Object-Oriented Programming
 8. Resources
 9. Acknowledgments
 10. Conclusion
-
-

Requirements

Prerequisites

- Python 3.10 or later
- Internet connection (to fetch data from TheMealDB)

Install Dependencies

The only external library required is requests. Install it with:

```
pip install requests
```

All other modules (socket, threading, json, sys, os) are part of the Python standard library.

File Structure

Make sure the three files are in the same folder:

```
project/  
■■■ server.py  
■■■ client1.py  
■■■ protocol.py
```

How To Run

Step 1 — Run the Server

first open a terminal then cd to the project folder

```
cd path/to/project
```

then run the server

```
python server.py
```

it will load the categories, areas and ingredients from themebldb and save them to reference_B4.json then start waiting for clients

Step 2 — Run the Client

open a new terminal and cd to the same folder again

```
cd path/to/project
```

then run the client

```
python client1.py
```

it will ask you to enter your username then connect to the server — you can open more than one terminal and run the client multiple times to test multiple clients at the same time

Step 3 — Use the Menu

once connected you will see the main menu

1. Browse Recipes
2. Reference Lists
3. Quit

pick 1 to search or filter recipes, pick 2 to see categories areas and ingredients, pick 3 to disconnect

The Scripts

`server.py`

The server is the backbone of the system. It starts by loading three reference lists from TheMealDB into memory (the startup cache), then opens a TCP socket and waits for clients. Each client is handled in its own thread.

Key functions:

Function	Description
<code>load_reference_cache()</code>	Fetches categories, areas, and ingredients from TheMealDB at startup and saves them to memory
<code>handle_client(conn, addr)</code>	Runs in a separate thread for each client; receives requests and sends back responses
<code>search_by_name(keyword)</code>	Searches TheMealDB by recipe name keyword; returns up to 15 results
<code>filter_by_category(category)</code>	Filters recipes by category; returns up to 15 results
<code>filter_by_area(area)</code>	Filters recipes by cuisine/area; returns up to 15 results
<code>filter_by_ingredient(ingredient)</code>	Filters recipes by ingredient; returns up to 15 results
<code>get_random_meal()</code>	Fetches one random recipe with full details
<code>get_meal_details_by_id(meal_id)</code>	Fetches full details of a specific recipe by ID
<code>save_recipe_file(client_name, option, data)</code>	Saves each recipe response to a JSON file for evaluation
<code>send_data(conn, data)</code>	Serializes a Python dict to JSON and sends it over the socket
<code>receive_data(conn)</code>	Reads from the socket until the newline delimiter and parses the JSON

Key classes (OOP):

```
class Cache:
    def load_cache(self):
        # Loads reference data into memory at startup

class RecipeAPI:
    def search(self, keyword): ...
    def filter_category(self, category): ...
    def random(self): ...
    # etc.

class Server:
    def __init__(self):
        self.cache = Cache()
```

```

        self.api = RecipeAPI()
    def start(self):
        self.cache.load_cache()
        start_server()

```

Packages used: socket, threading, json, requests

`client1.py`

The client connects to the server, sends the username, then enters a menu-driven loop. It validates user input before sending any request and displays results in a clean, organized format.

Key functions:

Function	Description
main()	Entry point: connects to server, sends username, runs the main menu loop
handle_recipes_menu(sock)	Handles options 1.1–1.6: search, filter, random
handle_reference_menu(sock)	Handles options 2.1–2.3: categories, areas, ingredients
display_recipe_list_and_ask(sock, response)	Shows the short recipe list and asks the user to pick one for full details
print_recipe_details(meal)	Prints full recipe info: name, category, area, ingredients, instructions, YouTube, so
display_reference_list(response, label)	Prints a flat reference list (categories / areas / ingredients)
pick_from_list(label, options)	Shows a numbered list and validates the user's selection
extract_name(item)	Extracts the display name from a reference dict (e.g. {"strCategory": "Beef"} → "Be

Packages used: socket, sys, os

`protocol.py`

A shared module imported by both the server and client. It defines all message type constants, permitted parameter values, and the two core communication helpers.

Constants:

```

REQ_CATEGORIES = "GET_CATEGORIES"
REQ_AREAS      = "GET_AREAS"
REQ_INGREDIENTS = "GET_INGREDIENTS"
REQ_SEARCH     = "SEARCH_NAME"
REQ_FILTER_CAT = "FILTER_CATEGORY"
REQ_FILTER_AREA = "FILTER_AREA"
REQ_FILTER_ING = "FILTER_INGREDIENT"
REQ_RANDOM     = "RANDOM_RECIPE"
REQ_DETAILS    = "GET_DETAILS"
REQ_QUIT       = "QUIT"

```

Key functions:

```

def send_message(sock, message: dict) -> None:
    """Converts dict to JSON, appends newline delimiter, sends over socket."""

def receive_message(sock) -> dict:
    """Reads from socket until newline delimiter, returns parsed dict."""

```

Packages used: json, socket

Additional Concept — Object-Oriented Programming

We chose **Object-Oriented Programming (OOP)** as our additional concept.

What is OOP?

OOP is a programming paradigm that organizes code around **classes** and **objects** rather than standalone functions. A class is a blueprint that defines data (attributes) and behavior (methods). Objects are instances of a class. OOP encourages clean separation of responsibilities, making code easier to read, maintain, and extend.

The four main principles of OOP are:

- **Encapsulation** — bundling data and methods together inside a class
- **Abstraction** — hiding implementation details behind a clean interface
- **Inheritance** — a class can inherit behavior from another class
- **Polymorphism** — different classes can share the same method name with different behavior

How We Applied OOP in `server.py`

We defined three classes with clearly separated responsibilities:

`Cache` — manages reference data

```
class Cache:
    def __init__(self):
        self.categories = []
        self.areas = []
        self.ingredients = []

    def load_cache(self):
        load_reference_cache()
        self.categories = reference_cache["categories"]
        self.areas = reference_cache["areas"]
        self.ingredients = reference_cache["ingredients"]
```

The Cache class is responsible for one thing only: holding and loading the static reference data. This follows the **Single Responsibility Principle**.

`RecipeAPI` — handles all TheMealDB API calls

```
class RecipeAPI:
    def search(self, keyword):
        return search_by_name(keyword)
    def filter_category(self, category):
        return filter_by_category(category)
    def filter_area(self, area):
        return filter_by_area(area)
    def filter_ingredient(self, ingredient):
        return filter_by_ingredient(ingredient)
    def random(self):
        return get_random_meal()
    def details(self, meal_id):
        return get_meal_details_by_id(meal_id)
```

RecipeAPI wraps all live API calls in one class. This means if the API changes in the future, only this class needs to be updated.

`Server` — coordinates everything

```
class Server:
    def __init__(self):
        self.cache = Cache()
        self.api = RecipeAPI()

    def start(self):
        self.cache.load_cache()
        start_server()
```

Server owns a Cache instance and a RecipeAPI instance. This is **composition** — building complex behavior by combining simpler objects. When the program runs, it creates a Server object and calls start():

```
if __name__ == "__main__":
    server = Server()
    server.start()
```

Class Relationships

```
Server
--> Cache      (composition)
--> RecipeAPI  (composition)
```

Resources

- ITNE352 Lecture Slides — Ch5: UDP and TCP
 - ITNE352 Lecture Slides — Ch6: Server Architecture and Multithreading
 - ITNE352 Lecture Slides — Ch7: HTTP and Python
 - ITNE352 Lecture Slides — Ch8: JSON and RESTful API
 - TheMealDB API Documentation
 - Python socket module
 - Python threading module
 - Python json module
 - Requests library
-

Acknowledgments

we would like to thank Dr. Mohammed Almeer for his help and guidance throughout this project, and TheMealDB for providing the free API we used.

Conclusion

in this project we learned how to run a client server system and how they communicate with each other. it also improved our python skills and helped us get better at using VS Code and GitHub. and we learned how to write a proper README file which was new for us.

